

Stacks: Overview and Objectives

An abstract data type built on last in, first out order

1. What this unit is about

A stack is an abstract data type, the same kind of construct as the linked list from the last unit. As with a linked list, we care about the wall between the operations a stack offers and how those operations get carried out behind the scenes, and a stack can be built on more than one foundation. Where the linked list unit compared array-based and pointer-based implementations of a list, this unit does the same comparison for a stack: an array-based version and a linked version, built on the node and link ideas you already know.

A stack holds its items in last in, first out order, often shortened to LIFO. Only one end of the stack, called the top, is available for adding or removing an item. The two core operations are push, which adds an item to the top, and pop, which removes and returns the item at the top. A third operation, peek or top, looks at the top item without removing it. Stacks show up constantly once you know to look for them: undo history in an editor, the back button in a browser, matching parentheses or brackets, evaluating an arithmetic expression, and the call stack that keeps track of recursive calls from the first unit of this course.

As with the earlier units, we work in **two languages at once**. Computer Science students write **C/C++**; CIS students write **Java**. The stack idea is identical in both, and both the array-based and linked implementations appear in each language.

Class time favors drawings and live code over slides. Read ahead, try tracing a push and pop sequence on paper yourself, and come ready to think out loud.

2. Learning objectives

By the end of the unit you should be able to:

Remember and understand

- State that a stack is an ADT, and recall the wall/information-hiding idea from the linked list unit as it applies to a stack.
- Define a stack and state its defining property, last in, first out order.
- Discuss the standard stack operations: push, pop, peek (or top), isEmpty, and size.
- Discuss the two implementations covered in this unit, array-based and linked, and recall the node and link vocabulary from the linked list unit as it applies to a linked stack.
- List common uses of a stack, including undo history, browser back buttons, matching parentheses or brackets, expression evaluation, and the recursive call stack.

Analyze

- Compare the array-based and linked implementations of a stack, including how each handles growth, memory use, and the cost of push and pop.
- Explain the relationship between the stack ADT and the run-time call stack used when a program executes a recursive function.
- Trace a stack by hand through a sequence of push, pop, and peek operations, and state its contents and top at each step.
- Judge whether a stack is the right ADT for a given problem, as opposed to a plain linked list or another structure already covered.

Apply

- Diagram a stack by hand, showing the top and the effect of a push or a pop.
- Code stack operations from scratch in both an array-based version and a linked version, in your language.
- Apply a stack to a standard problem, such as checking that parentheses or brackets are balanced, or evaluating a postfix expression.
- Generate a stack-based code solution to a new problem.

Evaluate

- Select the more appropriate stack implementation, array-based or linked, for a given scenario, and justify the choice against the alternative.